CMPE 252

Project 2: Neural Machine Translation with Attention
English to Spanish Transformer

May 5, 2026

Omar Jamjoum

Abstract

In this report, I go over the design, implementation, and evaluation of a neural machine translation (NMT) system that translates English to Spanish using a Transformer encoder-decoder architecture trained from scratch in PyTorch. The system uses multi-head self-attention and encoder–decoder cross-attention, following the Transformer architecture shown by Vaswani et al. (2017). The model was trained on ~200,000 sentence pairs sampled from the WMT News Commentary English to Spanish parallel corpus, tokenized with a shared SentencePiece BPE vocabulary of 16,000 subwords. Starting from a small baseline Transformer, I conducted systematic one variable at a time hyperparameter tuning experiments, to improve the baseline validation BLEU of 28.35 to a best validation BLEU of 35.85. The final configuration, using a wider model (d_model=512) with beam-search decoding (beam_size=5), achieved a corpus BLEU score of 36.50 on a fixed 2,000-example held-out test subset evaluated with SacreBLEU. While the project goal of working toward 80-85 BLEU was not reached, this result shows improvement with the constraints of a Colab scale training run and a scratch Transformer. The report documents the complete process, including data loading, preprocessing, training, evaluation, error analysis, and the challenges encountered.

1. Problem Statement

Machine translation is the task of automatically converting text from one natural language to another. Neural machine translation (NMT) systems based on the Transformer architecture have become the dominant approach in recent years, achieving strong results on many language pairs. The innovation of the Transformer is its reliance on attention mechanisms, specifically multi-head self-attention and encoder-decoder cross-attention. These mechanisms model dependencies between all positions in the input and output sequences without recurrence or convolution.

The objective of this project was to build an English to Spanish neural machine translation system using a Transformer encoder-decoder architecture trained from scratch in PyTorch, with attention as the central mechanism. The system was to be trained on official WMT parallel data, evaluated using SacreBLEU corpus BLEU, and iteratively improved through hyperparameter tuning. The goal was to work toward a BLEU score of 80-85, with the understanding that we wanted to show and document the improvement from the baseline experiment. The goal was a bit ambitious, given the constraints of a Colab-scale training environment.

Specific project requirements included: (1) use of WMT data for NMT with attention, (2) splitting data into at least train and test sets, (3) freedom to choose any suitable architecture, (4) inclusion of an attention mechanism, and (5) working toward the 80-85 BLEU goal with a documented improvement process starting from a baseline.

2. Methodology

2.1 Dataset

The training data used is from the WMT English to Spanish parallel corpus, specifically the *Statmt-news_commentary-16-eng-spa* (News Commentary v16) subset, accessed through the mtdata tool. This subset contained 369,540 raw sentence pairs of news-domain parallel text.

From the full corpus, 200,000 pairs were randomly sampled to fit within Colab memory and training time, while still conducting a fair experiment with relevant results. After whitespace normalization, length filtering (1 to 80 words per sentence), and exact-pair deduplication, 199,336 clean, unique sentence pairs remained. These were split into three non-overlapping partitions: 179,402 training pairs (90%), 9,967 validation pairs (5%), and 9,967 test pairs (5%). A data leakage check confirmed zero overlap between any pair of splits.

2.2 Tokenization

A shared SentencePiece BPE tokenizer was trained on the concatenation of English and Spanish training sentences. This produced a vocabulary of 16,000 subword units. Sharing the vocabulary across both languages allows the model to learn cross lingual subword representations and simplifies the architecture. Four special tokens were reserved: <pad> (ID 0), <unk> (ID 1), <s> / BOS (ID 2), and </s> / EOS (ID 3). Source sequences were capped at 128 subword tokens, and target sequences included BOS and EOS markers within the same 128-token limit.

2.3 Model Architecture

The model follows the standard Transformer encoder-decoder architecture from Vaswani et al. (2017), implemented using PyTorch's nn.Transformer module. This architecture uses multi-head self-attention in both encoder and decoder layers, and encoder-decoder cross-attention in the decoder.

Key architectural components:

Shared Embedding: A single nn.Embedding layer maps both source and target token IDs into the model's hidden space, since both languages share the SentencePiece vocabulary.

Positional Encoding: Sinusoidal positional encodings are added to the embedding outputs, following the original Transformer formulation, with dropout applied afterward.

Transformer Core: The nn.Transformer module provides the encoder stack, with self-attention, the decoder stack, with self-attention and cross-attention, layer normalization, and feed-forward sublayers.

Output Projection: A linear layer projects decoder hidden states to vocabulary logits for next-token prediction.

Xavier uniform initialization was applied to the embedding and output projection weights. The PAD embedding was zeroed out. A boolean causal mask was generated for autoregressive decoding, and key-padding masks were used to ignore PAD tokens in both encoder and decoder.

2.4 Baseline Configuration

The baseline model (Experiment 1) used the following hyperparameters: d_model=256, nhead=4, 2 encoder layers, 2 decoder layers, dim_feedforward=512, dropout=0.1, batch_size=64, learning_rate=1e-4, AdamW optimizer with Noam-style warm-up (4,000 steps) and inverse-square-root decay, CrossEntropyLoss with label_smoothing=0.1 (ignoring PAD), gradient clipping at 1.0, mixed-precision training (AMP), and greedy decoding. This configuration had ~10.8 million trainable parameters.

2.5 Training Procedure

Training used teacher forcing, where the decoder receives the ground-truth target prefix shifted by one position. Each epoch iterated over the full training set in mini-batches of 64 pairs, with gradients clipped at a max norm of 1.0. Mixed-precision training (torch.amp) was enabled on CUDA to accelerate computation. Validation loss was computed after each epoch, and the best checkpoint, which was determined by lowest validation loss, was saved to Google Drive. Early stopping with a patience of 3 epochs was available but not triggered in the final runs, which all trained for the full 10 epochs.

2.6 Hyperparameter Tuning Strategy

Tuning was conducted by changing one parameter at a time relative to the baseline, isolating the effect of each change. The tuning plan consisted of four experiments:

Exp	Change from Baseline	Parameter Isolated	Retraining?
1	Baseline (Section 2.4)	—	Yes (from scratch)
2	d_model 256 → 512	Model capacity	Yes (from scratch)
3	dropout 0.1 → 0.2	Regularization	Yes (from scratch)
4	beam_size=5 on Exp 2 model	Decoding strategy	No (decode only)

2.7 Decoding Strategies

Two decoding strategies were implemented. Greedy decoding selects the highest-probability token at each step and is used for all training experiments (Experiments 1 through 3). Beam search maintains the top-k hypotheses at each step with length normalization (alpha=0.6) and was applied as a decode-only experiment (Experiment 4) on the best trained model.

2.8 Evaluation Metric

All BLEU scores were computed using SacreBLEU (sacrebleu.corpus_bleu), for reproducible corpus level BLEU evaluation. Validation BLEU was measured on a fixed subset of 2,000 pairs drawn from the validation split, and the final test BLEU was measured on a separate fixed subset of 2,000 pairs from the held-out test split. Using fixed evaluation subsets ensures that BLEU comparisons across experiments are fair and reproducible.

3. Results

3.1 Experiment Summary

The following table summarizes the results of all four experiments. Each row reports the model configuration, training and validation loss at the final epoch, validation BLEU score on the fixed 2,000-pair subset, and wall-clock time.

Experiment	d_model	Dropout	Beam	Train Loss	Val Loss	Val BLEU	Time (min)
Exp1 Baseline	256	0.1	1	3.4292	3.2869	**28.35**	11.5
Exp2 d512	512	0.1	1	2.8911	2.9100	**34.35**	11.9
Exp3 drop02	256	0.2	1	3.8273	3.5777	**24.29**	11.6
Exp4 Beam	512	0.1	5	—	—	**35.85**	44.4 (decode)

Note: All experiments used 2 encoder layers, 2 decoder layers, 4 attention heads, dim_feedforward=512, lr=1e-4, and batch_size=64. Experiment 4 applied beam search to the Exp2 model without retraining.

3.2 Experiment Analysis

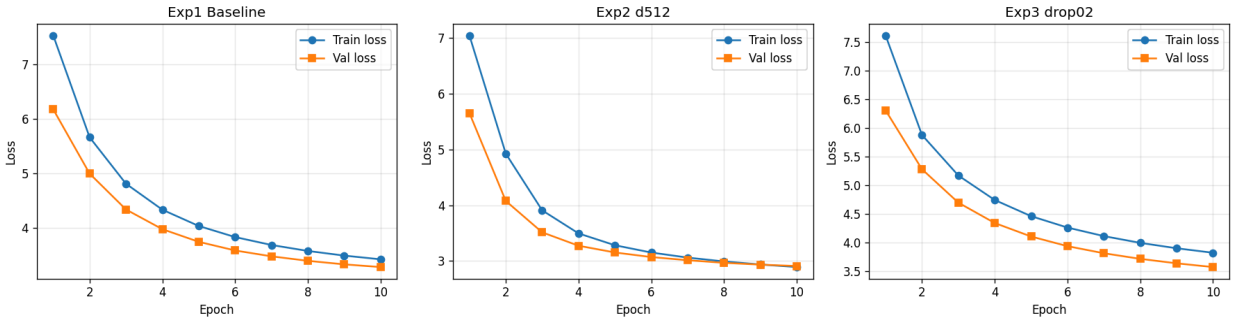

Figure 1. Training vs. validation loss curves for Experiments 1 through 3. All three models show steady convergence; Exp2 (d_model=512) achieves the lowest loss.

Experiment 1: Baseline

The baseline Transformer (d_model=256, ~10.8M parameters) trained for 10 epochs on an NVIDIA A100-SXM4-80GB GPU in ~11.5 minutes. Both training and validation loss decreased steadily across all epochs, with no sign of overfitting. The model achieved a validation BLEU of 28.35 with greedy decoding. Sample translations showed that the model could create grammatically coherent Spanish for common sentence structures. However, it did struggle with named entities, short headlines, and less frequent vocabulary.

Experiment 2: Larger Model Width (d_model=512)

Doubling the model width to d_model=512 (~24.8M parameters) produced the largest single improvement, raising validation BLEU from 28.35 to 34.35 (+6.00 points). Both

training and validation loss were lower than the baseline at every epoch. From this experiment we can infer that the baseline model was capacity-limited and that the additional parameters allowed the model to better capture translation patterns. Training time didn't increase much, to ~11.9 vs. ~11.5 minutes due to GPU parallelism.

Experiment 3: Higher Dropout (0.2)

Increasing dropout from 0.1 to 0.2 while keeping all other parameters at baseline values reduced validation BLEU to 24.29 (−4.06 points versus baseline). The higher regularization slowed learning and resulted in higher loss at every epoch. This experiment shows that the baseline dropout of 0.1 was already appropriate for this model size and dataset scale, and that additional regularization gave worse results in this configuration.

Experiment 4: Beam Search Decoding

Applying beam search, beam_size=5, length normalization alpha=0.6, to the best trained model, which was Experiment 2, improved validation BLEU from 34.35 to 35.85 (+1.50 points) without any retraining. This confirms that beam search provides a slight and consistent improvement over greedy decoding. This is because it explores a wider hypothesis space. The decode-only evaluation took ~44.4 minutes due to the sequential nature of beam search over 2,000 examples.

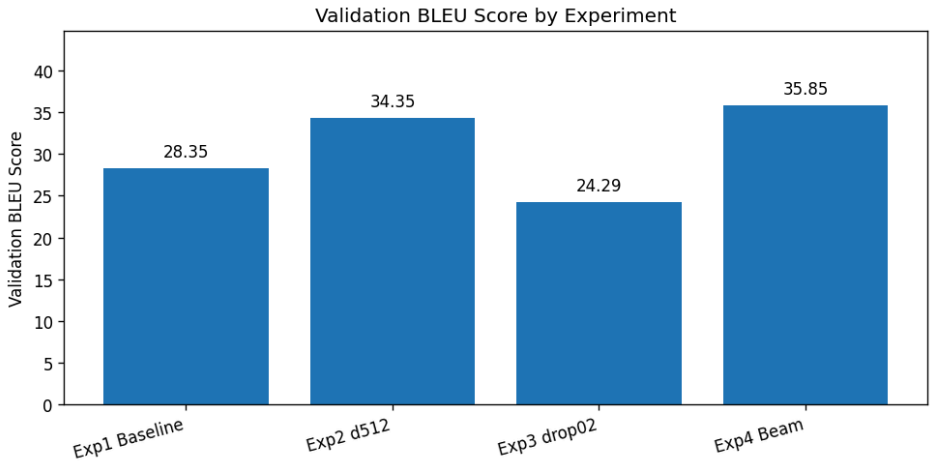

Figure 2. Validation BLEU score by experiment. Increasing model width (Exp2) and adding beam search (Exp4) yielded the largest gains; higher dropout (Exp3) hurt performance.

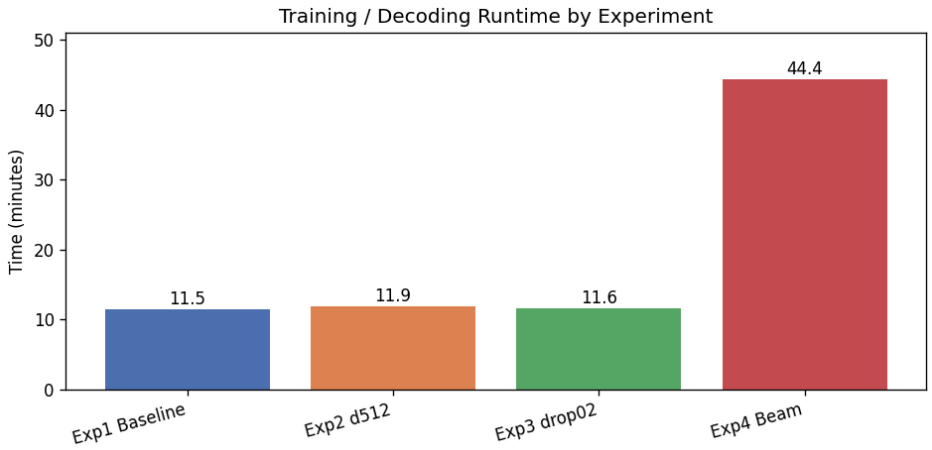

Figure 3. Training and decoding runtime by experiment. Training time was similar across Experiments 1 through 3 (~11-12 min); Exp4 beam-search decoding required 44.4 minutes.

3.3 Final Test Evaluation

The best configuration, which was Experiment 4: d_model=512 model with beam_size=5 decoding, was evaluated on a fixed 2,000-example held-out test subset that was never used during training or hyperparameter selection. The final test-subset BLEU score was 36.50. This result is consistent with the validation BLEU of 35.85. This finding confirms that the model generalizes to unseen data without significant degradation.

3.4 Error Analysis

A qualitative error analysis was conducted by examining individual translations sorted by sentence-level BLEU. The model produced exact, or near exact translations, for many common news domain sentences. The highest scoring examples included short factual statements and well represented constructions. The lowest scoring examples revealed several error patterns: (1) incomplete translations, particularly for short headline style inputs that the model truncated early; (2) named entity errors, where proper nouns, country names, and titles were copied, shortened, or translated awkwardly; (3) lexical choice errors, where the model selected a related but incorrect Spanish phrase; and (4) short-title instability, where very short inputs received disproportionately low sentence BLEU from a single missing word.

A scatter plot of sentence BLEU versus source sentence length showed no clear degradation for longer sentences. The average sentence BLEU for short inputs, 21 words or fewer, was 33.9. Comparing this with 34.9 for longer inputs, indicates that sentence length alone was not a primary driver of errors.

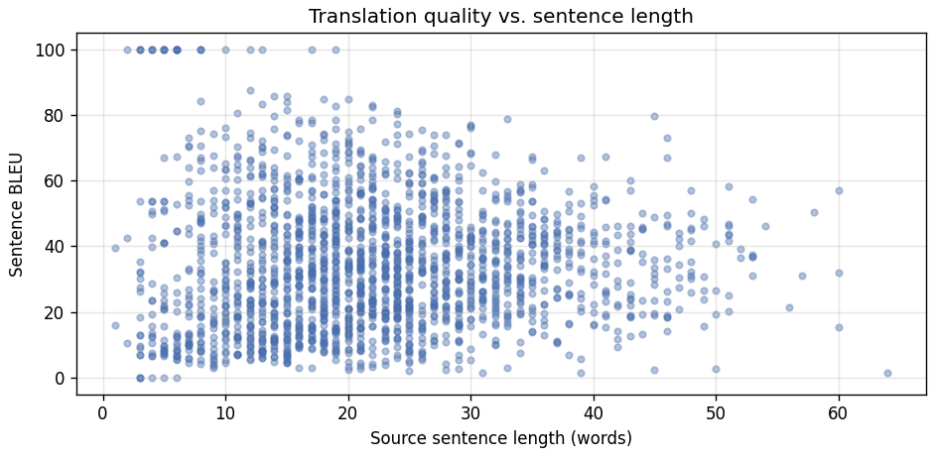

Figure 4. Sentence BLEU vs. source sentence length (test set). No clear degradation is observed for longer sentences; errors are driven more by named entities and headlines than by length.

3.5 Relationship to the 80-85 BLEU Goal

The project goal was to work toward a BLEU score of 80-85, and not to exactly reach it. The best final result was the test-subset BLEU of 36.50 using the Exp4 beam search configuration. This is expected and reasonable given the constraints of the project such as limited resources and training time. Achieving 80+ BLEU on general-domain English to Spanish translation typically requires substantially larger training, which may include tens of millions of sentence pairs, larger and deeper Transformer models, possibly 6+ layers, d_model=1024+, extensive hyperparameter search, advanced techniques such as back translation and data augmentation, pre-trained models or fine-tuning from large language models, and significantly more training time. The important result of this project is the documented improvement process which brought the results from a 28.35 baseline to a 36.50 final test BLEU.

4. Challenges and How We Overcame Them

4.1 Dataset Size and GPU Constraints

The full News Commentary v16 corpus contains 369,540 sentence pairs, and the broader WMT shared task includes millions more. However, Colab GPU memory and session time limits restricted training to ~200,000 sampled pairs. I addressed this by selecting a single but high quality sub set, which was News Commentary, sampling it to a manageable size, and caching processed data and checkpoints to Google Drive so that interrupted sessions could resume.

4.2 Avoiding Data Leakage

To prevent inflated evaluation metrics, the entire dataset was cleaned and deduplicated before splitting into train, validation, and test sets. This order is important because cleaning after splitting can create duplicates when normalization collapses distinct raw sentences into identical cleaned forms. I also used an automated leakage to confirm zero overlap between all three splits.

4.3 Tokenization and Sequence Length

Using a shared SentencePiece BPE tokenizer across both languages simplified the architecture but required careful handling of the vocabulary size and special tokens. The 16,000 subword vocabulary was a compromise between coverage and embedding-table size. Sequence length was capped at 128 subword tokens, which covered the vast majority of sentence pairs after word-level filtering to 80 words. Target sequences needed room for both BOS and EOS within this limit.

4.4 Beam Search Runtime

Beam search decoding with beam_size=5 took ~44 minutes for 2,000 test examples, compared with minutes for greedy decoding. This is because beam search performs multiple sequential decoder forward passes per token per hypothesis. I mitigated this by limiting BLEU evaluation to fixed 2,000-example subsets and by running beam search only on the best trained model rather than during every tuning iteration.

4.5 Reaching 80-85 BLEU

The most fundamental challenge was reaching 80-85 corpus BLEU since it is an extremely high expectation for a Transformer trained from scratch on 200,000 sentence pairs with only 10 epochs and a 2-layer architecture. NMT systems that reach or are close to these scores are usually state of the art and use magnitudes of more data, deeper models, advanced training techniques, and often pre-trained components. I

addressed this by focusing on demonstrating measurable, documented improvements through systematic tuning.

5. Conclusions

This project implemented a complete neural machine translation pipeline for English to Spanish translation using a Transformer encoder-decoder architecture trained from scratch in PyTorch. The system was trained on ~200,000 sentence pairs from the WMT News Commentary English to Spanish parallel corpus, tokenized with a shared 16,000 subword SentencePiece BPE vocabulary, and evaluated using SacreBLEU corpus BLEU.

Through systematic one variable at a time hyperparameter tuning, the model improved from a baseline validation BLEU of 28.35 to a best validation BLEU of 35.85. The final configuration, which used d_model=512 with beam search decoding, at beam_size=5, achieved a corpus BLEU of 36.50 on a fixed 2,000-example test subset. The important findings from the tuning process include that increasing model width (d_model) had the largest positive impact on translation quality, increasing the dropout beyond 0.1 hurt performance for this model and data scale, and beam search provided a consistent improvement over greedy decoding without requiring retraining.

While the 80-85 BLEU goal was not reached, validation BLEU improved by 7.50 points, from 28.35 to 35.85. The final selected model achieved 36.50 BLEU on the fixed held out test subset. The complete workflow, from official WMT data loading, cleaning, and deduplication, through Transformer training with attention, controlled tuning, validation-based model selection, held-out test evaluation, plotting, and error analysis, provides a solid foundation for future work with larger models and datasets.

6. References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS). arXiv:1706.03762.

[2] WMT / mtdata: https://www2.statmt.org/wmt24/mtdata/ — WMT machine translation shared-task data access tool.

[3] News Commentary Parallel Corpus: Statmt-news_commentary-16-eng-spa, accessed through mtdata.

[4] PyTorch nn.Transformer: https://pytorch.org/docs/stable/generated/torch.nn.Transformer.html

[5] Kudo, T. & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. https://github.com/google/sentencepiece

[6] Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. Proceedings of WMT. https://github.com/mjpost/sacrebleu

7. Appendix

A. Environment Details

Component	Version / Detail
Python	3.12.13
PyTorch	2.10.0+cu128
NumPy	2.0.2
SentencePiece	0.2.1
SacreBLEU	2.6.0
GPU	NVIDIA A100-SXM4-80GB (85.1 GB)
Random Seed	42

B. Dataset Statistics

Split	Pairs	EN Mean Words	ES Mean Words
Train	179,402	22.2	26.1
Validation	9,967	22.3	26.2
Test	9,967	22.3	26.2

C. Baseline Model Configuration

Hyperparameter	Value
vocab_size	16,000
d_model	256
nhead	4
num_encoder_layers	2
num_decoder_layers	2
dim_feedforward	512
dropout	0.1
learning_rate	1e-4
warmup_steps	4,000
epochs	10
label_smoothing	0.1
grad_clip	1.0
batch_size	64

max_seq_len	128
Parameters	10,844,800

D. Sample Translations from the Final Model

The following examples were generated by the final model configuration: Exp4 Beam, using d_model=512 and beam_size=5 on the fixed held-out test subset.

High-quality translation:

Source: The Gavi model is now under the microscope.

Reference: El modelo Gavi está ahora bajo el microscopio.

Hypothesis: El modelo Gavi está ahora bajo el microscopio.

Sentence BLEU: 100.0

Partial translation error:

Source: America Wakes Up to Climate Change

Reference: Los Estados Unidos despiertan al cambio climático

Hypothesis: EE. UU.

Sentence BLEU: 0.0

The first example shows that the model can produce exact translations for common sentence patterns. The second example shows a common failure mode: short headline-style inputs can be shortened too aggressively/early.

E. AI-Use Disclosure

Claude was used to help me research and draft portions of my project. AI helped me format and structure my notebook, and report, and fix various code bugs. I also used AI to improve markdown and report explanations, reviewing experiment design, and suggest and verify possible hyperparameter-tuning strategies. All code, outputs, interpretations, and conclusions were reviewed, verified, and executed by me.

F. Code Repository

The complete source code and additional files for this project are available at the following repository:

GitHub Repository: https://github.com/OmJam/NMT-NLP